

Encoding Arithmetic In First Order Logic (DRAFT)

Paul Vanderveen

July 2026

1 Introduction

The purpose of this document is to provide an intermediate representation (IR) for encoding arithmetic (i.e. $+$, $\times$, $=$, $<$, $>$) using first order logic (FOL) without the equality symbol $=$ or free variables. Furthermore, a first order logic statement of the form $\exists a \forall x \exists y \forall z \forall z' P(a, x, y, z, z')$ where P is quantifier free and only contains binary primitive predicates.

2 Overview

We construct a first order sentence (yet to be constructed) in the language of First Order Logic without equality and without bounded variables and without an arbitrary domain where the satisfying model is an IR of addition and multiplication (to do). We start out with a sentence of the form $\forall x \forall z \forall z' \in \mathbb{N} P(x, z, z', z'', z''')$ where P is quantifier free, equality is allowed, and we can reference arbitrary successors (i.e. $x+1$, $z+1$, $z'+1$, ...) and prove the correctness for the IR of addition. Afterwards, we prove the correctness of the IR for multiplication, before eliminating certain quantifiers, replacing equality symbol with auxillary predicates, and replacing successors with predicates and/or treating them as Skolemized existential variables. The final result is a first order sentence of the form $\exists a \forall x \exists y \forall z \forall z' P(a, x, y, z, z')$.

2.1 Construction Idea

The basic idea is to construct a model using only binary primitive predicates, and of the form $\forall \exists \forall *$ (signature of the first order sentence). Natively, addition and multiplication equations lend themselves to ternary predicate representations:

$$\text{Add}(a, b, c) \iff a + b = c$$

$$\text{Mult}(a, b, d) \iff a \times b = d$$

However, we cannot treat these ternary predicates as primitives per our problem statement. The idea instead is to look at the arguments of a binary predicates not as operands, but as ways to store data.

In particular, we treat the columns of a binary predicate (second argument) as a data index to retrieve information about addition and multiplication operations.

Every addition rule $a + b = c$ should be stored in some column n . Similarly, every multiplication rule $a \times b = d$ should also be stored in some column m . For convenience and efficiency, we can use the same column n to store the operands of both addition and multiplication. We introduce binary predicates A, B, C, D for this.

$$\text{Add}(a, b, c) \iff (\exists n : A(a, n) \wedge B(b, n) \wedge C(c, n))$$

$$\text{Mult}(a, b, d) \iff (\exists n : A(a, n) \wedge B(b, n) \wedge D(d, n))$$

So far, this guarantees every addition and multiplication operation can be "stored" in some respective column. Ideally, we would simply want to "compute" addition and multiplication by addressing the operands (a, b) and getting the result (i.e. $c = a + b$, $d = a \times b$). The simplest idea to enforce this is to ensure that every column has at least some stored value of a, b, c , and d . Furthermore, to prevent conflict and simplify the proof of correctness, we will want to assert that such a, b, c, d are unique. We can write this as :

$$\forall n, \exists_1 a, b, c A(a, n) \wedge B(b, n) \wedge C(c, n) \wedge D(d, n)$$

($\exists_1$ is the uniqueness quantifier)

This way, every $n \in \mathbb{N}$ corresponds to exactly one addition and multiplication rule with the same operands a and b .

3 Arithmetic in FOL without equality

The idea of the previous section was to store an infinite "additive" and "multiplicative" "lookup table" for addition and multiplication. However, we will need to construct an explicit mapping that can map operands (a, b) to a column index n . Furthermore, first order statement of the type AEA* naturally only allow one successor reference (i.e. $x \rightarrow x + 1$) by performing Skolemization on the existential quantifier. From a geometric standpoint, we can interpret the model structure of AEA* as behaving like a 2-dimensional grid in the first quadrant of the coordinate plane. The columns represent horizontal movements, and the rows represent vertical movements.

With this in mind, we want to construct a function, or more specifically, a bijection (since $\mathbb{N}^2$ and $\mathbb{N}$ are both countable) where shifting (± 1) the operands moves to the next column (to the right) as much as possible. Let f define the mapping of a, b to its index column n .

3.1 Bijections

Let $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ be a function from the set $\mathbb{N}^2$ to $\mathbb{N}$ defined by:

$$f(a, b) = \binom{a + b + 1}{2} = \frac{(a + b)^2 + (a + b)}{2} + a = \frac{a^2 + 2ab + b^2 + a + b}{2} + a$$

The first few mappings defined by f are:

$$\begin{aligned} f(0, 0) &: 0 \\ f(0, 1) &: 1 \\ f(1, 0) &: 2 \\ f(0, 2) &: 3 \\ f(1, 1) &: 4 \\ f(2, 0) &: 5 \\ f(0, 3) &: 6 \\ f(1, 2) &: 7 \\ f(2, 1) &: 8 \\ f(3, 0) &: 9 \\ f(0, 4) &: 10 \end{aligned}$$

etc...

Intuitively, this appears to be a bijection from $\mathbb{N}^2 \rightarrow \mathbb{N}$. In fact we will prove this because this is a crucial step in proving the correctness of an IR model of addition and multiplication that is constructed later.

Theorem 3.1. *f is a bijective function from $\mathbb{N}^2$ to $\mathbb{N}$.*

Proof. To show f is a bijection from $\mathbb{N}^2$ to $\mathbb{N}$. We must show that:

- f is onto (all natural numbers have the form $(a^2 + 2ab + b^2 + a + b)/2 + a$)
- f is one-to-one (all distinct pairs (a, b) map to different values)

We will show that f is onto.

By the well ordering principle, for every natural number n , there exists a smallest k such that

$$n < \binom{k+1}{2}$$

. Because k is the smallest integer such that $n < \binom{k+1}{2}$, it must be the case that $\binom{k}{2} \leq n$.

Because $\binom{k}{2} \leq n$, there is a non-negative integer r such that $\binom{k}{2} + r = n$.

Since $0 \leq n < \binom{k+1}{2}$, we have:

$$\begin{aligned} 0 &< \binom{k+1}{2} \\ 0 &< \frac{k^2 + k}{2} \\ 0 &< k^2 + k \\ -k &< k^2 \end{aligned}$$

If k is a natural number, then the equality holds for all $k > 0$. Hence, $k \geq 1$.

We have from earlier:

$$\binom{k}{2} + r = n$$

So

$$\begin{aligned} \binom{k}{2} + r &< \binom{k+1}{2} \\ r &< \binom{k+1}{2} - \binom{k}{2} \\ r &< k \end{aligned}$$

(which follows Pascal's Identity)

Because a and b were defined to be arbitrary natural numbers, and $k \geq 1$, we can assert the existence of some a and b such that $a + b + 1 = k$. Because r was an integer less than k (or $\leq k - 1$), we have $r < a + b + 1$, or $r \leq a + b$. Since a and b are arbitrary numbers that sum to $k - 1$, and $r \leq k - 1$, we can, without loss of generality, let $r = a$.

Hence, we have shown the existence of integers a and b such that for any n , $n = \binom{a+b+1}{2} + a$.

We now need to show that for any pairs of integers (a, b) and (c, d) , $(f(a, b) = f(c, d)) \implies (a = c) \wedge (b = d)$ (i.e. injectivity).

Suppose that $f(a, b) = f(c, d) = n \in \mathbb{N}$.

We have $\binom{a+b+1}{2} + a = \binom{c+d+1}{2} + c$.

Let $u = a + b + 1$ and $v = c + d + 1$. We have:

$$\binom{u}{2} + a = \binom{v}{2} + c$$

Without loss of generality (by the well ordering principle), suppose that $u \leq v$. Let $u + w = v$ for some nonnegative integer w .

We have $a < u$ and $u \geq 1$. This, along with Pascal's Triangle Identity (which implies $\binom{u+1}{2} > \binom{u}{2}$ since $u > 0$), yields:

$$\binom{u}{2} + a \leq \binom{u}{2} + a + b (= \binom{u}{2} + u - 1) < \binom{u+1}{2} \leq \binom{u+i}{2}$$

(for any $i > 0$)

Since $u + w = v$ and $\binom{u}{2} + a = \binom{u+w}{2} + a$, but $\binom{u}{2} + a < \binom{u+i}{2} + a$ for $i > 0$, w must be 0, so $u = v$, and $a + b = c + d$.

Hence,

$$\binom{u}{2} + a = \binom{v}{2} + c = n \implies a = c$$

and

$$a + b = c + d \implies b = d$$

Since f is both onto and one-to-one or injective, f must be a bijection from $\mathbb{N}^2$ to $\mathbb{N}$. $\square$

4 IR of Addition in First Order Logic

To encode addition rules using binary predicates, we rely on the construction methods described in Section 2, in conjunction with the bijection f we outlined in Section 3. The idea is to use diagonal top-down (for the first operand a) and bottom-up (for the second operand b) encodings, to translate between different addition rules between successive columns (i.e. n to $n + 1$). When the bottom up rule (i.e. b) approaches zero (the first row), we then "slide" the sum c to the next value $c + 1$ and reset the top down counter a to zero, while setting b to $c + 1$, the the top-down and bottom-up loops repeat. Because a starts at 0 and traverses to c , and b starts at c and traverses to zero, and c consistently always shifts by one, every summation rule $a + b = c$ is uniquely addressable in a certain column. For instance, Column 0 might address the rule $0 + 0 = 0$ while column 4 might address the rule $1 + 1 = 2$.

4.1 The First Order Statement

Statement 4.1.

$$\forall x \forall z \forall z' \in \mathbb{N} :$$

$$\begin{aligned} & A(0, 0) \wedge \\ & B(0, 0) \wedge \\ & C(0, 0) \wedge \\ & ((A(z, x) \wedge A(w, x)) \implies z = z') \wedge \\ & ((B(z, x) \wedge B(w, x)) \implies z = z') \wedge \\ & ((C(z, x) \wedge C(w, x)) \implies z = z') \wedge \\ & (C(x, z) \implies (C(x, z+1) \vee C(x+1, z+1))) \wedge \\ & ((C(x, z) \wedge A(x, z)) \implies \neg A(x+1, z+1)) \wedge \\ & ((\neg C(x, z) \wedge A(x, z)) \implies A(x+1, z+1)) \wedge \\ & (B(x+1, z) \implies B(x, z+1)) \wedge \\ & (B(0, x) \implies A(0, x+1)) \wedge \\ & ((B(0, x) \wedge C(z, x)) \implies \neg C(z, x+1)) \wedge \\ & ((\neg B(0, x) \wedge C(z, x)) \implies C(z, x+1)) \wedge \\ & ((A(0, x) \wedge C(z, x)) \implies B(z, x)) \wedge \end{aligned}$$

Theorem 4.1. *For all natural numbers n , there exists natural numbers a, b, c such that $a + b = c$ and $A(a, n) \wedge B(b, n) \wedge C(c, n)$. Furthermore, such a, b, c are unique.*

Proof. The theorem must be proven for all eligible pairs (n, a, b, c) . To make the proof more efficient, we invoke the at-most-one rules in Statement 4.1, effectively showing the uniqueness of a, b , and c such that $A(a, n) \wedge B(b, n) \wedge C(c, n)$:

$$\begin{aligned} & ((A(z, n) \wedge A(z', n)) \implies z = z') \wedge \\ & ((B(z, n) \wedge B(z', n)) \implies z = z') \wedge \\ & ((C(z, n) \wedge C(z', n)) \implies z = z') \end{aligned}$$

We can prove the existence criteria (that a, b, c actually exist), and the sum relationship ($a + b = c$) by induction. The remainder of the proof proceeds by induction on n . Suppose $S(n)$ is the statement we want to prove for all $n \in \mathbb{N}$:

$$\begin{aligned} S(n) = \\ \exists a, b, c : A(a, n) \wedge B(b, n) \wedge C(c, n) \end{aligned}$$

and

$$(A(a, n) \wedge B(b, n) \wedge C(c, n)) \implies (a + b = c)$$

The initial case $S(0)$ is trivially met by the axioms of Statement 4.1 (where $n, a, b, c = 0$):

$$A(0, 0) \wedge B(0, 0) \wedge C(0, 0)$$

and

$$a + b = c \text{ is true : } 0 + 0 = 0$$

For the induction on n , Suppose that $S(n)$ is true. Let (a, b, c) be the unique pair satisfying $S(n)$ [inductive hypothesis]. Let (a', b', c') be the unique pair satisfying $S(n + 1)$, if it exists. We divide the proof into two cases based on whether or not $B(0, n)$ is true or false.

First assume $B(0, n)$ is false. Because we have the existence of a b such that $B(b, n)$ it must be the case that $b > 0$, and so the rule (letting $x + 1 = b$):

$$B(b, z) \implies B(b - 1, z + 1)$$

applies (as $b - 1$ is nonnegative). So $B(b - 1, n + 1)$ is true. Additionally (from the falsity of $B(0, x)$), the rule

$$((\neg B(0, n) \wedge C(c, n)) \implies C(c, n + 1))$$

also applies (letting $x = n$ and $z = c$). Hence, $C(c, n + 1)$ is true.

Now letting $x = a$ and $z = n$, we obtain the following two implications:

$$((C(a, n) \wedge A(a, n)) \implies \neg A(a + 1, n + 1))$$

$$((\neg C(a, n) \wedge A(a, n)) \implies A(a + 1, n + 1))$$

If $C(a, n)$ were true, then $c = a$ by uniqueness constraints, and $a + b = c = a$, which requires $b = 0$, which contradicts that $b > 0$. Therefore, $C(a, n)$ must be false (and $a \neq c$), in which case, the second implication rule applies, while the first is vacuously true. By the second rule, we have $A(a + 1, n + 1)$ to be true. By letting $a' = a + 1$, $b' = b - 1$, $c' = c$ have found a pair (a', b', c') such that:

$$A(a', n + 1) \wedge B(b', n + 1) \wedge C(c', n + 1)$$

and

$$a' + b' = c' \text{ is true : } a' + b' = a + 1 + b - 1 = a + b = c = c'$$

From the uniqueness of a', b', c' , no such other pairs exist and the statement S holds for $n + 1$ when $B(0, n)$ is false.

Now assume, for the second case, that $B(0, n)$ is true. In that case, it follows directly from the rule

$$B(0, n) \implies A(0, n + 1)$$

in Statement 4.1 that $A(0, n + 1)$ is also true. By the rule (substituting $x = n$ and $z = c$ in the original statement):

$$(B(0, n) \wedge C(c, n)) \implies \neg C(c, n + 1)$$

and by the rule

$$C(c, n) \implies (C(c, n + 1) \vee C(c + 1, n + 1))$$

we can conclude that since $C(c, n + 1)$ is false (first rule listed above), $C(c + 1, n + 1)$ must be true (second rule listed above). Using the rule ($z = c + 1$, $x = n + 1$):

$$(A(0, n + 1) \wedge C(c + 1, n + 1)) \implies B(c + 1, n + 1)$$

in conjunction with the previously stated two, we can conclude that $B(c + 1, n + 1)$ is true. Hence we have the existence of elements $a' = 0$, $b' = c + 1$, and $c' = c + 1$ such that:

$$A(a', n + 1) \wedge B(b', n + 1) \wedge C(c', n + 1)$$

and

$$a' + b' = c' \text{ is true : } 0 + (c + 1) = c + 1$$

By the at most one constraint of a', b', c' , $S(n + 1)$ must therefore be true (since there are no other (a', b', c') pairs to prove).

Therefore, since the statement $S(n + 1)$ is true in both cases (depending on whether $B(0, n)$ is true or false), it holds. $\square$

Theorem 4.2. *For all natural numbers n, a, b , $A(a, n)$ and $B(b, n)$ are true if and only if $n = f(a, b)$.*

Proof. In the previous theorem, we established that there exists a unique a and b such $A(a, n)$ and $B(b, n)$ are true, for every n . By Theorem 3.1, because f is a bijection, there must exist unique u and v such $n = f(u, v)$. The goal is to show that $u = a$ and $v = b$.

We can prove this by induction:

$$S(n) = \{\forall a, b, u, v \in \mathbb{N} ((f(u, v) = n) \wedge A(a, n) \wedge B(b, n)) \implies ((u = a) \wedge (v = b))\}$$

For $S(0)$, we previously established that $(a, b, c) = (0, 0, 0)$ is the unique pair (a, b, c) .

On the other hand if $f(u, v) = \binom{u+v+1}{2} + u = \frac{u^2+2uv+v^2+u+v}{2} + u$ is zero, then $u = 0$ and $v^2 + v = 0 \implies v = 0$. Hence $u = a = 0$ and $v = b = 0$.

For the inductive step, we can, similar to the previous theorem, split the step into two cases.

The inductive hypothesis assumes that $u = b$ and $v = a$.

As with the previous proof let (a, b, u, v) be the pair that satisfies $S(n)$. Let $c = a + b$. Let (a', b') be the pair satisfying $A(a', n+1) \wedge B(b', n+1)$, $c' = a' + b'$, and (u', v') be the pair satisfying $f(u', v') = n+1$. Both (a', b') and (u', v') must exist by Theorems 4.1 and 3.1, respectively. The objective is to show they are equal.

Case I: $b \neq 0$.

In this case, since b is a natural number, $b > 0$. It was shown in Theorem 4.1 that for $n+1$, both $A(a+1, n+1)$ and $B(b-1, n+1)$ are both true. Hence $a' = a+1$ and $b' = b-1$. Because of the uniqueness of solutions to $f(u', v') = n+1$, it is sufficient to show that $f(a+1, b-1) = n$. (which would imply that $u = a' = a+1$ and $v = b' = b-1$).

Using the binomial form:

$$f(a+1, b-1) = \binom{a+1+b-1+1}{2} + a+1 = \binom{a+b+1}{2} + a+1 =$$

$$\left(\binom{a+b+1}{2} + a \right) + 1 =$$

$$n+1$$

Hence, since $f(a+1, b-1) = n+1$, $u = a' = a+1$ and $v = b' = b-1$ and the induction step is proven.

Case II : $b = 0$.

In this case, we have $f(a, 0) = \binom{a+1}{2} + a = \frac{a^2+a}{2} + a = \frac{a^2+3a}{2} = n$. From the inductive reasoning of Theorem 4.1, it was shown that for $n+1$, $A(0, n+1)$ and $B(c+1, n+1)$, and consequently $C(c+1, n+1)$ is true ($a' = 0, b' = c+1, c' = c+1$). As $f(u', v') = n+1$, it is sufficient to show that $f(0, c+1) = f(0, a+b+1)$ evaluates to $n+1$:

$$f(0, c+1) = \binom{0+(c+1)+1}{2} + 0 = \binom{c+2}{2}$$

Yet by the induction Hypothesis, $f(a, b) = \binom{a+b+1}{2} + a = \binom{c+1}{2} + a = n$.

Using Pascal's Identity, we have:

$$\binom{c+2}{2} - \binom{c+1}{2} = c+1 = a+b+1$$

So we can express

$$\begin{aligned} \binom{c+2}{2} &= \\ \binom{c+1}{2} + c+1 &= \\ \binom{a+b+1}{2} + a+b+1 &= \\ \binom{a+1}{2} + a+1 &= \\ (\text{because } b=0) & \\ \left(\binom{a+1}{2} + a \right) + 1 &= \\ f(a,b) + 1 &= \\ n+1 & \end{aligned}$$

(applying the inductive hypothesis)

Hence, $f(0, c+1) = \binom{c+2}{2} = n+1$, so $u' = a' = 0$ and $v' = b' = c+1$.

In either case ($b \neq 0$) or $b = 0$, the inductive step holds and $S(n+1)$ is true. $\square$

Corollary 4.1. *For all natural numbers a, b , there exist exactly one natural number n such that $A(a, n)$, $B(b, n)$, and consequently, $C(a+b, n)$ are all true.*

This directly follows from the reasoning Theorem 4.1 to show they exist, the reasoning of Theorem 4.2 to establish the relationship of a, b to f , and Theorems 3.1 and 4.1 to show that such a and b are unique, and again, Theorem 4.1 to show that if $C(c, n)$ is true, $c = a+b$.

5 IR of Multiplication in First Order Logic

In the addition representation, we were able to diagonally "shift" the operands a and b in opposite directions to enumerate all integers a, b that sum to any given integer c , and when the enumeration was done (i.e. $b = 0$), we increment c and repeat the process forever.

Unfortunately, multiplicative properties are difficult to enumerate in a sequential column fashion. Instead, now that we have additive properties established for A , B , and C , we can use those to encode multiplication only using recursive

addition rules. The idea is to use the fact that $a \times b$ is equivalent to adding a , b number of times. The recursive solution is as follows:

$$\begin{cases} a \times b = 0, & \text{if } b = 0 \\ a \times b = a \times (b - 1) + a, & \text{if } b > 0 \end{cases}$$

To implement this behavior in first order logic, we objectively want to encode the property

$$(A(a, n) \wedge B(b, n) \wedge D(d, n)) \implies a \times b = d$$

By Theorem 4.2, we already have the operands a and b encoded into a unique column n (and their sum c), so the only remaining objective is to encode their product d , into the same column. As with A , B , C , we will want to establish uniqueness for D . We will also need an intermediate variable P to represent the previous product (i.e. $p = a \times (b - 1)$) and then address the column which computes the sum of a and p , and then assert that the product of a and b is equal to the sum of a and p . We describe the first order statement below. We use z_2, z_3, z_4 as extra quantifiers instead of the extra z' quantifier (variable naming convention change). The conjunctions contained in the first set of square brackets are the addition rules we proved hold in Section 4. The conjunctions contained in the second set of square brackets are the multiplication rules we intend to encode and prove hold as intended.

5.1 The First Order Statement

Statement 5.1.

$$\forall x \forall z \forall z_2 \forall z_3 \forall z_4 \in \mathbb{N} :$$

$$\begin{aligned}
& [A(0,0) \wedge \\
& B(0,0) \wedge \\
& C(0,0) \wedge \\
& ((A(z,x) \wedge A(z_2,x)) \implies z = z_2) \wedge \\
& ((B(z,x) \wedge B(z_2,x)) \implies z = z_2) \wedge \\
& ((C(z,x) \wedge C(z_2,x)) \implies z = z_2) \wedge \\
& (C(x,z) \implies (C(x,z+1) \vee C(x+1,z+1))) \wedge \\
& ((C(x,z) \wedge A(x,z)) \implies \neg A(x+1,z+1)) \wedge \\
& ((\neg C(x,z) \wedge A(x,z)) \implies A(x+1,z+1)) \wedge \\
& (B(x+1,z) \implies B(x,z+1)) \wedge \\
& (B(0,x) \implies A(0,x+1)) \wedge \\
& ((B(0,x) \wedge C(z,x)) \implies \neg C(z,x+1)) \wedge \\
& ((\neg B(0,x) \wedge C(z,x)) \implies C(z,x+1)) \wedge \\
& ((A(0,x) \wedge C(z,x)) \implies B(z,x))] \\
& \wedge \\
& [((D(z,x) \wedge D(z_2,x)) \implies z = z_2) \wedge \\
& ((P(z,x) \wedge P(z_2,x)) \implies z = z_2) \wedge \\
& (B(0,x) \implies D(0,x)) \wedge \\
& (B(0,x) \implies \neg P(z,x)) \wedge \\
& ((A(z,z_3) \wedge B(x,z_3) \wedge D(z_2,z_3) \wedge A(z,z_4) \wedge B(x+1,z_4)) \implies P(z_2,z_4)) \wedge \\
& ((A(x,z_2) \wedge P(z,z_2) \wedge A(x,z_3) \wedge B(z,z_3) \wedge C(z_4,z_3)) \implies D(z_4,z_2))]
\end{aligned}$$

Theorem 5.1. *For all natural numbers n , there exists natural numbers a, b, d such that $a \times b = d$ and $A(a, n) \wedge B(b, n) \wedge D(d, n)$. Furthermore, such a, b, d are unique.*

Proof. From Theorem 4.1, we already know there exists a and b such that $A(a, n) \wedge B(b, n)$ are both true. Therefore, the only remaining objective is to show that $D(d, n)$ is true iff $a \times b = d$.

It helps to establish the uniqueness rules for D and P ahead of time:

$$\begin{aligned}
& ((D(z, x) \wedge D(z_2, x)) \implies z = z_2) \\
& ((P(z, x) \wedge P(z_2, x)) \implies z = z_2)
\end{aligned}$$

Hence, we now only have to show that for this specific d , it exists, and is the product of a and b .

Rather than inducting in the columns as we did for the proof of theorem 4.1, we will instead induct on the value of b , treating a as an arbitrary fixed integer. Let S be the following statement:

$$S(b) : (\forall a \forall d (A(a, n) \wedge B(b, n) \wedge D(d, n)) \implies (a \times b = d))$$

$S(0)$ reduces down to $(\forall a \forall d (A(a, n) \wedge B(0, n) \wedge D(d, n)) \implies (a \times 0 = d))$
so for the statement to hold, $d = 0$. We have the implication rule

$$(B(0, n) \implies D(0, n))$$

(substituting $x = n$) so for the implication to follow, $d = 0$ (which is true if $b = 0$) and the consequent $a \times 0 = 0$ is true, so $S(0)$ is true. If $d \neq 0$, then $D(d, n)$ is false and $S(0)$ vacuously true. In either case, $S(0)$ is true. Now assume $S(b)$ is true. For a fixed $b > 0$ (we just showed $b = 0$ yields a true statement), we have the rule

$$((A(z, z_3) \wedge B(x, z_3) \wedge D(z_2, z_3) \wedge A(z, z_4) \wedge B(x + 1, z_4)) \implies P(z_2, z_4))$$

Making the substitutions $z = a$, $x = b$, $z_2 = d$, $z_3 = n$, $z_4 = m$, this becomes:

$$((A(a, n) \wedge B(b, n) \wedge D(d, n) \wedge A(a, m) \wedge B(b + 1, m)) \implies P(d, m))$$

By the induction hypothesis, $a \times b = d$. The unique column m exists operands a and $b + 1$, and we directly assert $P(a \times b, m)$ to be true. Since true values of P (for each column) are unique, we know that P uniquely encodes the previous product into the row addressing the successor of the second operand $b + 1$. By the rule

$$((A(x, z_2) \wedge P(z, z_2) \wedge A(x, z_3) \wedge B(z, z_3) \wedge C(z_4, z_3)) \implies D(z_4, z_2))$$

Making the substitutions $x = a$, $z = d$, $z_2 = m$, $z_3 = m'$, $z_4 = a + d$, this becomes:

$$((A(a, m) \wedge P(d, m) \wedge A(a, m') \wedge B(d, m') \wedge C(a + d, m')) \implies D(a + d, m))$$

From the previous rule and deduction steps, $P(d, n)$ was implied to be true, $A(a, m)$ was true by assumption, there is a unique column m' containing operands a and d , respectively, and their sum $a + d$ (predicate C). Hence $D(a + d, m)$ is true.

Because m was assumed to be the unique column variable holding a and $b + 1$ as operands (i.e. $A(a, m)$ and $B(b + 1, m)$), and $a + d = a + a \times b = a \times (b + 1)$, it follows that the statement $S(b + 1)$ is true (since the value of d' such that $D(d', m)$ is true, is the product of a and $b + 1$).

□

In later sections [TODO], we will describe how to rewrite statement 5.1 using less quantifiers (quantifier elimination), at the cost of only adding a

few more intermediate predicates, eliminating equality comparisons (uniqueness constraints), eliminating successor references to $z + 1$, and eliminating the use of constant 0 (other than $(0, 0)$).

6 Equality, Successor, and Constant Eliminations

In this section, we turn our attention to the statements that make use of equality, successor references (other than x), and constant zero (other than $(0, 0)$). When we mention our First-Order statement of interest, we will be referring to Statement 5.1.

The uniqueness constraints we imposed are:

$$\begin{aligned} (A(z, x) \wedge A(z_2, x)) &\implies z = z_2 \\ (B(z, x) \wedge B(z_2, x)) &\implies z = z_2 \\ (C(z, x) \wedge C(z_2, x)) &\implies z = z_2 \\ (D(z, x) \wedge D(z_2, x)) &\implies z = z_2 \\ (P(z, x) \wedge P(z_2, x)) &\implies z = z_2 \end{aligned}$$

The statements making references to successor $z + 1$ are:

$$\begin{aligned} C(x, z) &\implies (C(x, z + 1) \vee C(x + 1, z + 1)) \\ (C(x, z) \wedge A(x, z)) &\implies \neg A(x + 1, z + 1) \\ (\neg C(x, z) \wedge A(x, z)) &\implies A(x + 1, z + 1) \\ B(x + 1, z) &\implies B(x, z + 1) \end{aligned}$$

The statements that make use of the constant 0 other than the point of origin are:

$$\begin{aligned} B(0, x) &\implies A(0, x + 1) \\ (B(0, x) \wedge C(z, x)) &\implies \neg C(z, x + 1) \\ (\neg B(0, x) \wedge C(z, x)) &\implies C(z, x + 1) \\ (A(0, x) \wedge C(z, x)) &\implies B(z, x) \\ B(0, x) &\implies D(0, x) \\ B(0, x) &\implies \neg P(z, x) \end{aligned}$$

There are a few reasons why we want to re-write such statements that avoid using certain constants or symbols.

From an objective standpoint, the quantifier prefix form AEA^* ($\forall\exists\forall*$) only allows for at most one successor reference, that is, Skolemizing the existential quantifier and binding it as a function of the first universal statement. The structure does not permit that for all universal variables. Hence, we must get rid of terms like $z + 1$ or find another way to preserve the structure.

Furthermore, equality is strictly forbidden in the classical version Entscheidungsproblem [REF NEEDED!!!]. The only reason equality is needed is to enforce uniqueness of values of A, B, C, D and P for each column, so enforcing the uniqueness constraint of say, row values of a such that $A(a, n)$ is true, but $a \neq a' \implies \neg A(a', n)$, is sufficient enough.

The reason we want to eliminate the constant zero (other than $(0, 0)$) is that the AEA^* prefix does not allow for constant references. While in theory, we may assert that we have a non-empty domain via EAEA^* , the first existential only serves that purpose, and is not intended to be used elsewhere other than fixing points at the origin.

6.1 Equality elimination

In this subsection, we will turn our attention to the elimination of the equality variable $=$ in statements like

$$(A(z, x) \wedge A(z_2, x)) \implies z = z_2$$

Before explaining the solution, it makes sense to first explain the general idea. Suppose we have a unary predicate $\mathbb{P}$ (not to confuse with P). The standard way to express that any true value for $\mathbb{P}$ is:

$$\forall x, z : (\mathbb{P}(x) \wedge \mathbb{P}(z)) \implies (x = z)$$

This is also what is called as an "at most one" constraint, because the statement is still true if no value of x makes $\mathbb{P}$ true, and true if there is exactly one value of x that makes $\mathbb{P}$ true.

To bypass using the equality sign, we introduce an auxiliary predicate $\mathbb{P}'$, which acts as a "switch" that toggles when the first value of x such that $\mathbb{P}(x)$ is true occurs.

$$\forall x, z : \neg(\mathbb{P}(x) \wedge \mathbb{P}'(x)) \wedge (\mathbb{P}(x) \implies \mathbb{P}'(x+1)) \wedge (\mathbb{P}'(x) \implies \mathbb{P}'(x+1))$$

By forcing $\mathbb{P}$ and $\mathbb{P}'$ to be mutually exclusive, and because every $n > x$ has $\mathbb{P}'(n)$ to be true, it forces $\mathbb{P}(n)$ to be false, meaning once $\mathbb{P}(x)$ occurs as a true value for some x , it can never occur again (at most one constraint). Alternatively, one

can count "downwards" (towards zero) and achieve the same result (the same reasoning works, except $n < x$):

$$\forall x, z : \neg(\mathbb{P}(x) \wedge \mathbb{P}'(x)) \wedge (\mathbb{P}(x+1) \implies \mathbb{P}'(x)) \wedge (\mathbb{P}'(x+1) \implies \mathbb{P}'(x))$$

We may wish to use the latter set of implications for most cases, because it will make things more efficient and compact later on.

6.2 Successor elimination

In the previous section, we showed how we can eliminate equality symbols by enforcing uniqueness through auxiliary "switch" predicates.

Enforcing the successor behavior (specifically targeting $z+1$) without directly referencing it requires more auxiliary predicates to complete a two-step process. Initially, we have statements such as

$$C(x, z) \implies (C(x, z+1) \vee C(x+1, z+1))$$

that we want to re-write without the $z+1$ reference. We explain below how to eliminate that reference. Suppose we have a coordinate (x, z) and we want to "travel" to (or more technically, reference), $(x+1, z+1)$. We cannot reference that coordinate in one shot while not referencing $z+1$. Instead, we replace the reference mapping:

$$(x, z) \rightarrow (x+1, z+1)$$

with:

$$(x, z) \rightarrow (x+1, z)$$

$$(x+1, z) \rightarrow (x+1, z+1)$$

However, rather than writing the second leap as

$$(x+1, z) \rightarrow (x+1, z+1)$$

, we can simply write it as

$$(z, x) \rightarrow (z, x+1)$$

because we are universally quantifying over x and z .

Like the previous section, we will use $\mathbb{P}$ to explain the process, though we will treat $\mathbb{P}(x)$ as a binary predicate instead. Suppose we have:

$$\forall x, z : \mathbb{P}(x, z) \implies \mathbb{P}(x+1, z+1)$$

We will introduce the auxiliary predicate $\mathbb{P}^*$ to refer to the connecting step (in the two step process) to shift the second argument by one, and then finally, the first argument by one.

$$\forall x, z : (\mathbb{P}(x, z) \implies \mathbb{P}^*(x+1, z)) \wedge (\mathbb{P}^*(z, x) \implies \mathbb{P}(z, x+1))$$

Because x, z are universal, we can plug in, say, if $\mathbb{P}(u, v)$ is true, $x = v$ and $z = u$ which leads to $\mathbb{P}^*(u+1, v)$ from the first implication, and then plug in $x = v$ and $z = u+1$, to get $\mathbb{P}(u+1, v+1)$ from the second implication. It is, of course, worth noting that the same approach works regardless of whether we shifted rows first, columns first, or wanted to move leftwards (i.e. $(x-1, z-1)$). In fact, all four statements imply the same logic, just in different "directions":

$$\begin{aligned} \forall x, z : (\mathbb{P}(x, z) \implies \mathbb{P}^*(x+1, z)) \wedge (\mathbb{P}^*(z, x) \implies \mathbb{P}(z, x+1)) \\ \forall x, z : (\mathbb{P}(z, x) \implies \mathbb{P}^*(z, x+1)) \wedge (\mathbb{P}^*(x, z) \implies \mathbb{P}(x+1, z)) \\ \forall x, z : (\mathbb{P}(x+1, z) \implies \mathbb{P}^*(x, z)) \wedge (\mathbb{P}^*(z, x+1) \implies \mathbb{P}(z, x)) \\ \forall x, z : (\mathbb{P}(z, x+1) \implies \mathbb{P}^*(z, x)) \wedge (\mathbb{P}^*(x+1, z) \implies \mathbb{P}(x, z)) \end{aligned}$$

6.3 Non Origin Constant elimination

In this section, we intend to remove or rewrite references to zero such as

$$B(0, x) \implies A(0, x+1)$$

that are not the point of origin (i.e. like $A(0, 0)$). The solution to rewriting something like $B(0, x) \implies A(0, x+1)$ without 0 as the first argument is to create a binary predicate Z which is true iff the first argument is zero.

For example, if we had something like :

$$\forall x : \mathbb{P}(0, x)$$

we introduce a predicate Z such that:

$$\forall x, z : Z(0, 0) \wedge \neg Z(x+1, z) \wedge (Z(z, x) \implies Z(z, x+1))$$

This way, Z is true only along the first row (because it is false all successor rows, and true values propagate along rows where it is true once, i.e. at the origin, it propagates horizontally). We can directly replace the constant zero (with predicate $\mathbb{P}$ as):

$$\forall x, z : Z(z, x) \implies \mathbb{P}(z, x)$$

The implication only yields a consequence if the first argument (i.e. z) is zero, preserving logical equivalence of $\mathbb{P}$. It is important to note, that in ideal scenarios, we would want to make as few references to zero as possible. This is useful for unit propagation and reducing the number of disjunctions and quantifiers a clause has. This is something we will focus on in later section when we re-write sentences with constant zero.

7 Rewriting the Arithmetic Sentence

In this section, we apply the structural equivalents discussed in Sections 6.1, 6.2, and 6.3 to the sentence described in 5.1. Re-writing or re expressing the conjunctions or clauses requires careful preservation of existing predicate behavior. We have to handle A, B, C, D, P specially to avoid introducing conflicting states or behaviors.

7.1 Replacing Equality

We proceed by replacing the statements

$$\begin{aligned} (A(z, x) \wedge A(z_2, x)) &\implies z = z_2 \\ (B(z, x) \wedge B(z_2, x)) &\implies z = z_2 \\ (C(z, x) \wedge C(z_2, x)) &\implies z = z_2 \\ (D(z, x) \wedge D(z_2, x)) &\implies z = z_2 \\ (P(z, x) \wedge P(z_2, x)) &\implies z = z_2 \end{aligned}$$

with

$$\begin{aligned}
& \neg(A(x, z) \wedge A'(x, z)) \\
& A(x + 1, z) \implies A'(x, z) \\
& A'(x + 1, z) \implies A'(x, z) \\
& \neg(B(x, z) \wedge B'(x, z)) \\
& B(x + 1, z) \implies B'(x, z) \\
& B'(x + 1, z) \implies B'(x, z) \\
& \neg(C(x, z) \wedge C'(x, z)) \\
& C(x + 1, z) \implies C'(x, z) \\
& C'(x + 1, z) \implies C'(x, z) \\
& \neg(D(x, z) \wedge D'(x, z)) \\
& D(x + 1, z) \implies D'(x, z) \\
& D'(x + 1, z) \implies D'(x, z) \\
& \neg(P(x, z) \wedge P'(x, z)) \\
& P(x + 1, z) \implies P'(x, z) \\
& P'(x + 1, z) \implies P'(x, z)
\end{aligned}$$

to establish uniqueness criteria.

7.2 Replacing Multiple successors

In this subsection, we look at applying the principles in Section 6.2 to re-write certain statements that make use of the successor $z + 1$. In particular, the statements of consideration are:

$$\begin{aligned}
C(x, z) & \implies (C(x, z + 1) \vee C(x + 1, z + 1)) \\
(C(x, z) \wedge A(x, z)) & \implies \neg A(x + 1, z + 1) \\
(\neg C(x, z) \wedge A(x, z)) & \implies A(x + 1, z + 1) \\
B(x + 1, z) & \implies B(x, z + 1)
\end{aligned}$$

We will focus on the diagonal escalating behavior for predicates A and B first, which are more straightforward. We might be tempted to write out $(A(x, z) \implies A^*(x + 1, z)) \wedge (A^*(z, x) \implies A(z, x + 1))$. However, the main issue is that this creates an infinite escalation, but the implication rules stop the escalation once the same coordinate is true for A and C .

To preserve the structure, we can focus on preventing the consequence of the first implication from occurring if $C(z, x)$ is also true.

$$\begin{aligned}
((C(x, z) \wedge A(x, z)) \implies \neg A^*(x+1, z)) \wedge \\
((\neg C(x, z) \wedge A(x, z)) \implies A^*(x+1, z)) \wedge \\
(A^*(z, x) \implies A(z, x+1))
\end{aligned}$$

This way, the diagonal escalation (top down) stops as soon as A and C have the same row value for a given column. For the diagonal shift pattern involving B , we can use a similar pattern, however, we will switch columns first before switching rows, to keep the auxiliary predicate "below" the original predicate B , which might be useful for future applications of this sentence construction.

$$\begin{aligned}
(B(z, x) \implies B^*(z, x+1)) \wedge \\
(B^*(x+1, z) \implies B(x, z))
\end{aligned}$$

It is worth briefly explaining why we do not need to, say, enforce disjointness (i.e. between A and A^* , and again, between B and B^*) or uniqueness with diagonal auxiliary predicates. In particular, the deterministic implication $(A^*(z, x) \implies A(z, x+1))$ implies that if there are two distinct values a, a' such that $A^*(a, n)$ and $A^*(a', n)$ are both true, then $A(a, n+1)$ and $A(a', n+1)$ will both be true, violating the uniqueness of the A predicate (for every column). The uniqueness of A' follows from A .

Similarly, if there are two non-zero distinct values b, b' such that $B^*(b, n)$ and $B^*(b', n)$ are simultaneously true, then the deterministic rule $(B^*(x+1, z) \implies B(x, z))$ forces $B(b-1, n)$ and $B(b'-1, n)$ to simultaneously be true, violating the uniqueness of B . If one of b, b' , on the other hand (for change of assumption) is zero, then the implication $B^*(x+1, z) \implies B(x, z)$ doesn't apply. The non-zero uniqueness of B^* follows from the uniqueness of B . The fact that $B^*(0, b)$ is true does not change the underlying behavior of B .

In both cases, the behavior of A and B remains preserved.

We now turn our attention to the diagonal behavior of C , which is primarily captured with:

$$C(x, z) \implies (C(x, z+1) \vee C(x+1, z+1))$$

We introduce the auxiliary step predicate C^* as follows:

$$\begin{aligned}
(C(z, x) \implies (C(z, x+1) \vee C^*(z, x+1)) \wedge \\
(C^*(x, z) \implies C(x+1, z))
\end{aligned}$$

(note, we switched x and z in the original implication to avoid referencing $z+1$)
 Like the case with A^* and B^* , we do not need to enforce uniqueness rules for C^* . In fact, it very much follows the same reasoning as the uniqueness of A . Disjointedness is automatically implied: if c is a natural number such that $C(c, n)$ and $C^*(c, n)$ are both true, then $C(c+1, n)$ follows from the implication $(C^*(x, z) \implies C(x+1, z))$ and the uniqueness of C is contradicted.

7.3 Replacing and minimizing Zero References

In this subsection, we look at applying the principles in Section 6.3, along with minimizing references to zero (even if not explicit), reducing quantifier variable usage, and clause (number of disjuncts) lengths, which makes unit propagation easier and avoids constant-like behavior. We defined Z as:

$$\forall x, z : Z(0, 0) \wedge \neg Z(x+1, z) \wedge (Z(z, x) \implies Z(z, x+1))$$

The statements we want to replace are:

$$\begin{aligned} B(0, x) &\implies A(0, x+1) \\ (B(0, x) \wedge C(z, x)) &\implies \neg C(z, x+1) \\ (\neg B(0, x) \wedge C(z, x)) &\implies C(z, x+1) \\ (A(0, x) \wedge C(z, x)) &\implies B(z, x) \\ B(0, x) &\implies D(0, x) \\ B(0, x) &\implies \neg P(z, x) \end{aligned}$$

To avoid using extra quantifiers, we will define another predicate W , which will be true for all rows in a given column n if $B(0, n)$ is true, and false for all rows otherwise if $B(0, n)$ is false.

$$\begin{aligned} ((Z(z, x) \wedge B(z, x)) &\implies W(z, x)) \wedge \\ ((Z(z, x) \wedge \neg B(z, x)) &\implies \neg W(z, x)) \wedge \\ (W(x, z) &\iff W(x+1, z)) \end{aligned}$$

This allows us to conveniently replace $B(0, x)$ with $W(z, x)$, which we can do as:

$$\begin{aligned}
& ((W(z, x) \wedge B(z, x)) \implies A(z, x + 1)) \wedge \\
& ((W(z, x) \wedge C(z, x)) \implies \neg C(z, x + 1)) \wedge \\
& ((\neg W(z, x) \wedge C(z, x)) \implies C(z, x + 1))
\end{aligned}$$

For the statement $(A(0, x) \wedge C(z, x)) \implies B(z, x)$, we do not have a way to distinguish a column where $A(0, x)$ is true from one where it isn't. Instead, it is useful to infer that the previous column has $B(0, x)$ true (by forward propagation) and consequently, $W(z, x)$ is true. Then $C(z, x+1)$ is false, so $C^*(z, x+1)$ is true, so $C(z+1, x+1)$ is true. Of course, we must write this from the perspective of the column where $A(0, x)$ is true, so in terms of that, $C^*(x, z) \rightarrow C(x+1, z)$, and we can simply assert that $B(x+1, z)$ should also be true. We can then write that statement as:

$$C^*(x, z) \implies B(x+1, z)$$

The remaining two statements we have to re-write can be re-written as:

$$\begin{aligned}
& ((W(z, x) \wedge B(z, x)) \implies D(z, x)) \wedge \\
& (W(z, x) \implies \neg P(z, x))
\end{aligned}$$

7.4 Putting it together

With the statements in the previous subsection, we can re-write Statement 5.1 as:

Statement 7.1.

$$\forall x \forall z \forall z_2 \forall z_3 \forall z_4 \in \mathbb{N} :$$

$$\begin{aligned}
& [Z(0,0) \wedge \\
& \quad A(0,0) \wedge \\
& \quad B(0,0) \wedge \\
& \quad C(0,0) \wedge \\
& \quad \neg Z(x+1,z) \wedge \\
& \quad (Z(z,x) \implies Z(z,x+1)) \wedge \\
& \quad \neg(A(x,z) \wedge A'(x,z)) \wedge \\
& \quad (A(x+1,z) \implies A'(x,z)) \wedge \\
& \quad (A'(x+1,z) \implies A'(x,z)) \wedge \\
& \quad \neg(B(x,z) \wedge B'(x,z)) \wedge \\
& \quad (B(x+1,z) \implies B'(x,z)) \wedge \\
& \quad (B'(x+1,z) \implies B'(x,z)) \wedge \\
& \quad \neg(C(x,z) \wedge C'(x,z)) \wedge \\
& \quad (C(x+1,z) \implies C'(x,z)) \wedge \\
& \quad (C'(x+1,z) \implies C'(x,z)) \wedge \\
& \quad (C(z,x) \implies (C(z,x+1) \vee C^*(z,x+1)) \wedge \\
& \quad \quad (C^*(x,z) \implies C(x+1,z)) \wedge \\
& \quad ((C(x,z) \wedge A(x,z)) \implies \neg A^*(x+1,z)) \wedge \\
& \quad ((\neg C(x,z) \wedge A(x,z)) \implies A^*(x+1,z)) \wedge \\
& \quad (A^*(z,x) \implies A(z,x+1)) \wedge \\
& \quad (B(z,x) \implies B^*(z,x+1)) \wedge \\
& \quad (B^*(x+1,z) \implies B(x,z)) \wedge \\
& \quad ((Z(z,x) \wedge B(z,x)) \implies W(z,x)) \wedge \\
& \quad ((Z(z,x) \wedge \neg B(z,x)) \implies \neg W(z,x)) \wedge \\
& \quad (W(x,z) \iff W(x+1,z)) \wedge \\
& \quad ((W(z,x) \wedge B(z,x)) \implies A(z,x+1)) \wedge \\
& \quad ((W(z,x) \wedge C(z,x)) \implies \neg C(z,x+1)) \wedge \\
& \quad ((\neg W(z,x) \wedge C(z,x)) \implies C(z,x+1)) \wedge \\
& \quad (C^*(x,z) \implies B(x+1,z))]
\end{aligned}$$

$$\begin{aligned}
& \wedge \\
& [\neg(D(x, z) \wedge D'(x, z)) \wedge \\
& (D(x+1, z) \implies D'(x, z)) \wedge \\
& (D'(x+1, z) \implies D'(x, z)) \wedge \\
& \neg(P(x, z) \wedge P'(x, z)) \wedge \\
& (P(x+1, z) \implies P'(x, z)) \wedge \\
& (P'(x+1, z) \implies P'(x, z)) \wedge \\
& ((W(z, x) \wedge B(z, x)) \implies D(z, x)) \wedge \\
& (W(z, x) \implies \neg P(z, x)) \wedge \\
& ((A(z, z_3) \wedge B(x, z_3) \wedge D(z_2, z_3) \wedge A(z, z_4) \wedge B(x+1, z_4)) \implies P(z_2, z_4)) \wedge \\
& ((A(x, z_2) \wedge P(z, z_2) \wedge A(x, z_3) \wedge B(z, z_3) \wedge C(z_4, z_3)) \implies D(z_4, z_2))]
\end{aligned}$$

8 Eliminating extra quantifiers

In this section, we describe how to eliminate the z_3 and z_4 quantifiers by rewriting the logical statements

$$((A(z, z_3) \wedge B(x, z_3) \wedge D(z_2, z_3) \wedge A(z, z_4) \wedge B(x+1, z_4)) \implies P(z_2, z_4))$$

and

$$((A(x, z_2) \wedge P(z, z_2) \wedge A(x, z_3) \wedge B(z, z_3) \wedge C(z_4, z_3)) \implies D(z_4, z_2))$$

There are good practical reasons why we may want to do this. In satisfiability (SAT) solving, shorter clauses may lead to faster unit propagation. Having fewer universal variables leads to faster iterations and more straightforward inferences.

8.1 Eliminating the first quantifier

To understand how to eliminate the last two quantifiers (z_3 and z_4), it is good idea to understand the logic behind the recursive multiplicative statements. Regarding the first rule, all we are doing is retrieving the column n that contains the product rule $a \times b = d$, and storing that result in the column m containing a and $b+1$, as the previous product p (i.e. from the perspective of m , $p = a \times b$).

In the second rule, we look at an arbitrary column n (fresh reset here since we are talking about a global rule), and if there is a value p stored in that column (from the perspective of n), it has the value of $a \times (b-1)$, and we then retrieve the column m , which contains the operands a and $(b-1)p$ (so for column

m , the operands are a and p), and retrieve the sum $(c =)a + p$. We then store this result in column n as the designated d value $d = a + p$, which is equivalent to $d = a + a \times (b - 1) = a \times b$.

In summary, if we keep track locally of variables:

Let $A(a, n), B(b, n), D(d, n)$ be true.

So $a \times b = d$ is true.

Let m be the column containing $A(a, m)$ and $B(b + 1, m)$ to be true.

We then store the value of d , $(D(d, n))$ as the previous result in column m , as $P(d, m)$.

Let n' be the column containing $A(a, n')$ and $B(d, n')$ as true values. Then we retrieve the sum, $a + d$ ($C(a + d, n')$), and finally, we store this result back in column m as the product of $a + d = a \times (b + 1)$, so $D(a + d, m)$ is true.

The idea with eliminating quantifiers is to preserve this same logic, keeping A, B, C, D, P logically intact, while using extra auxiliary steps to retrieve the "locations" of intermediate operations. Looking at the antecedent of the first statement

$$(A(z, z_3) \wedge B(x, z_3) \wedge D(z_2, z_3) \wedge A(z, z_4) \wedge B(x + 1, z_4)) \implies P(z_2, z_4)$$

we do not have enough quantifiers to retrieve A, B, D and the column containing $b + 1$ simultaneously. Ultimately, we are trying to relate the column $f(a, b)$ to $f(a, b + 1)$.

Instead of trying to write a massive 6-clause statement to do it all simultaneously, the basic idea of relating the two columns of interest is to narrow down the search by relating the different columns that share the same a operands (i.e. n and m are related if $n = f(a, b)$ and $m = f(a, b')$), and similarly, the same idea applies if both columns have a b operand differing by one (i.e. n and m are related if $n = f(a, b)$ and $m = f(a', b + 1)$). Taking the intersection of these two relationships yields the exact column relationship we want. We introduce new sentences and auxiliary predicates R_1, R_2, R_3 below:

$$\begin{aligned} (A(x, z) \implies (A(x, z_2) \iff R_1(z_2, z))) \wedge \\ (B(x, z) \implies (B(x + 1, z_2) \iff R_2(z_2, z))) \wedge \\ ((R_1(x, z) \wedge R_2(x, z)) \iff R_3(x, z)) \wedge \\ ((D(x, z) \wedge R_3(z_2, z)) \implies P(x, z_2)) \end{aligned}$$

To see why these set of statements are equisatisfiable to $(A(z, z_3) \wedge B(x, z_3) \wedge D(z_2, z_3) \wedge A(z, z_4) \wedge B(x + 1, z_4)) \implies P(z_2, z_4)$ and the logical relationship between D and P is unchanged. To see why, consider the first of these statements:

$$A(x, z) \implies (A(x, z_2) \iff R_1(z_2, z))$$

It is not hard to show R_1 is an equivalence relation. However, simply showing that, from the perspective of z , R_1 filters out all columns that don't have the same a operand value is sufficient.

For each natural number n , there is exactly one a such that $A(a, n)$. For all other $a' \neq a$, $A(a', n)$ is false, and so the implication $A(x, z) \implies (A(x, z_2) \iff R_1(z_2, z))$ is vacuously true for $x = a'$. But for $x = a$, the antecedent is true, so the consequent $A(x, z_2) \iff R_1(z_2, z)$ must then be true. Because $A(a, n)$ is true, if $A(a, m)$ is true, then if $z = n$ and $z_2 = m$, it follows that $R_1(m, n)$ is true. If $A(a, m)$ is false, then so is $R_1(m, n)$. Hence, R_1 acts as a "filter" that for each column n , the rows $R_1(m, n)$ is true precisely when (if and only if) $n = f(a, b)$ and $m = f(a, b')$ for some b' .

The same reasoning applies to $B(x, z) \implies (B(x + 1, z_2) \iff R_2(z_2, z))$. While R_2 may not be an equivalence relation, for each column n , the exact values of m such that R_2 is true are those such that, assuming $n = f(a, b)$, $m = f(a', b + 1)$ for some a' .

Finally, R_3 acts as the intersection of the two subsets of rows filtered out by R_1 and R_2 , so it perfectly describes the relationship between $f(a, b)$ and $f(a, b + 1)$. i.e. for $n = f(a, b)$, $R_3(m, n)$ acts as a reference to the column $m = f(a, b + 1)$. The last implication then asserts that the current product encoded by $D(d = a \times b, n = f(a, b))$, d should be the previous product from the perspective of column $m = f(a, b + 1)$. That is, if the current product encoded by $D(d, n)$ and next row m , encoded by $R_3(m, n)$ are referenced, then store that value d as the previous product in column m , encoded by $P(d, m)$.

We use the auxillary predicates U_1 and U_2 to preserve equisatisfiability and behavior of the second clause

$$((A(x, z_2) \wedge P(z, z_2) \wedge A(x, z_3) \wedge B(z, z_3) \wedge C(z_4, z_3)) \implies D(z_4, z_2))$$

Like the first statement, we use auxillary predicate U_1 to filter rows that match the relationship of matching b operands and previous products p (i.e. if $n = f(a, p)$, $m = f(a, b)$, then n and m are related), and then take the intersection of that relationship with the relationship for common a -values (R_1), which the predicate U_2 captures. The sentences that encode this behavior are:

$$\begin{aligned} & (W(x, z) \implies \neg U_1(x, z)) \wedge \\ & (P(x, z) \implies (B(x, z_2) \iff U_1(z_2, z))) \wedge \\ & ((R_1(x, z) \wedge U_1(x, z)) \iff U_2(x, z)) \wedge \\ & ((C(x, z) \wedge U_2(z, z_2)) \implies D(x, z_2)) \end{aligned}$$

The explanation behind the first statement is that when the column of interest corresponds to $b = 0$, there is no previous product (recall $(W(z, x) \implies \neg P(z, x))$) because that case serves as a base case. Therefore, there should be

no auxiliary retrieval predicate either.

The second implication forces U_1 to be true only if another column (which is treated as a row from the current column's perspective) has a b value that matches the value of p such that $P(p, n)$ is true, n being the current column.

The intersection, described by the third implication and U_2 , of these filtered columns with those with matching a values yields the exact relation of n and m where $n = f(a, p)$ and $m = f(a, b)$. This allows, as described by the fourth statement, us to address the sum of $p = a \times b$ and a by retrieving c from column m , and then mark that value as the next product in column n by $D(a \times b + a, n)$, successfully capturing the recursive nature of multiplication as $a \times (b = 0) = 0$ and $a \times b = a \times (b - 1) + a$ if $b > 0$.

8.2 Putting it together

With the statements in the previous subsection, we can re-write Statement 7.1 as:

Statement 8.1.

$$\forall x \forall z \forall z_2 \in \mathbb{N} :$$

$$\begin{aligned}
& [Z(0,0) \wedge \\
& \quad A(0,0) \wedge \\
& \quad B(0,0) \wedge \\
& \quad C(0,0) \wedge \\
& \quad \neg Z(x+1,z) \wedge \\
& \quad (Z(z,x) \implies Z(z,x+1)) \wedge \\
& \quad \neg(A(x,z) \wedge A'(x,z)) \wedge \\
& \quad (A(x+1,z) \implies A'(x,z)) \wedge \\
& \quad (A'(x+1,z) \implies A'(x,z)) \wedge \\
& \quad \neg(B(x,z) \wedge B'(x,z)) \wedge \\
& \quad (B(x+1,z) \implies B'(x,z)) \wedge \\
& \quad (B'(x+1,z) \implies B'(x,z)) \wedge \\
& \quad \neg(C(x,z) \wedge C'(x,z)) \wedge \\
& \quad (C(x+1,z) \implies C'(x,z)) \wedge \\
& \quad (C'(x+1,z) \implies C'(x,z)) \wedge \\
& \quad (C(z,x) \implies (C(z,x+1) \vee C^*(z,x+1)) \wedge \\
& \quad \quad (C^*(x,z) \implies C(x+1,z)) \wedge \\
& \quad ((C(x,z) \wedge A(x,z)) \implies \neg A^*(x+1,z)) \wedge \\
& \quad ((\neg C(x,z) \wedge A(x,z)) \implies A^*(x+1,z)) \wedge \\
& \quad (A^*(z,x) \implies A(z,x+1)) \wedge \\
& \quad (B(z,x) \implies B^*(z,x+1)) \wedge \\
& \quad (B^*(x+1,z) \implies B(x,z)) \wedge \\
& \quad ((Z(z,x) \wedge B(z,x)) \implies W(z,x)) \wedge \\
& \quad ((Z(z,x) \wedge \neg B(z,x)) \implies \neg W(z,x)) \wedge \\
& \quad (W(x,z) \iff W(x+1,z)) \wedge \\
& \quad ((W(z,x) \wedge B(z,x)) \implies A(z,x+1)) \wedge \\
& \quad ((W(z,x) \wedge C(z,x)) \implies \neg C(z,x+1)) \wedge \\
& \quad ((\neg W(z,x) \wedge C(z,x)) \implies C(z,x+1)) \wedge \\
& \quad (C^*(x,z) \implies B(x+1,z))]
\end{aligned}$$

$$\begin{aligned}
& \wedge \\
& [\neg(D(x, z) \wedge D'(x, z)) \wedge \\
& (D(x+1, z) \implies D'(x, z)) \wedge \\
& (D'(x+1, z) \implies D'(x, z)) \wedge \\
& \neg(P(x, z) \wedge P'(x, z)) \wedge \\
& (P(x+1, z) \implies P'(x, z)) \wedge \\
& (P'(x+1, z) \implies P'(x, z)) \wedge \\
& ((W(z, x) \wedge B(z, x)) \implies D(z, x)) \wedge \\
& (W(z, x) \implies \neg P(z, x)) \wedge \\
& (A(x, z) \implies (A(x, z_2) \iff R_1(z_2, z))) \wedge \\
& (B(x, z) \implies (B(x+1, z_2) \iff R_2(z_2, z))) \wedge \\
& ((R_1(x, z) \wedge R_2(x, z)) \iff R_3(x, z)) \wedge \\
& ((D(x, z) \wedge R_3(z_2, z)) \implies P(x, z_2)) \wedge \\
& (W(x, z) \implies \neg U_1(x, z)) \wedge \\
& (P(x, z) \implies (B(x, z_2) \iff U_1(z_2, z))) \wedge \\
& ((R_1(x, z) \wedge U_1(x, z)) \iff U_2(x, z)) \wedge \\
& ((C(x, z) \wedge U_2(z, z_2)) \implies D(x, z_2))]
\end{aligned}$$

8.3 Uniqueness of D and P

In this subsection, we make a slight optimization to Statement 8.1 by removing the auxiliary predicates D' and P' and their associated statements, and enforcing the uniqueness constraint on D and P an alternative way, only adding and modifying a few more small clauses in exchange.

First, consider the implication clause:

$$(W(z, x) \wedge B(z, x)) \implies D(z, x)$$

Because W was defined earlier to be true for all rows in a specific column iff $B(0, x)$ is true for that column (i.e. $(Z(z, x) \wedge B(z, x))$), we can tether the uniqueness of D for $b = 0$ to the uniqueness of b by re-writing the above statement as :

$$W(z, x) \implies (B(z, x) \iff D(z, x))$$

the uniqueness of the resulting previous product (i.e. P has the previous value of D by increment b to $b+1$) can be obtained by modifying the clause:

$$(D(x, z) \wedge R_3(z_2, z)) \implies P(x, z_2)$$

to

$$R_3(z_2, z) \implies (D(x, z) \iff P(x, z_2))$$

Because R_3 is unique (i.e. by the uniqueness of a and $b + 1$ relative to a and b), and if the true value of D is unique, then so is the resulting row value that makes P true.

The last three remaining clauses:

$$\begin{aligned} (P(x, z) \implies (B(x, z_2) \iff U_1(z_2, z))) \\ ((R_1(x, z) \wedge U_1(x, z)) \iff U_2(x, z)) \\ ((C(x, z) \wedge U_2(z, z_2)) \implies D(x, z_2)) \end{aligned}$$

require that any value v such that $U_1(v, z)$ is true must also be relate p and b by the same row value. Since the row value of p is unique, that implies that for any two distinct columns v, v' such that $U_1(v, z)$ and $U_1(v', z)$, the b operand must be the same and match that of p .

Those such rows are then filtered out by R_1 , which is true for two columns $R_1(u, v)$ if and only if u and v have the same a operand, hence forcing the a and b operands to be unique, implying that U_2 is unique for each column. If we modify the last clause to:

$$U_2(z, z_2) \implies (C(x, z) \iff D(x, z_2))$$

Then the uniqueness of the next product (i.e. next d for the column containing a and $b + 1$) follows from the uniqueness of C . This kind of inductive-style reasoning effectively deems the uniqueness constraints on D and P unnecessary. With that in mind, the final sentence (plus re-phrasing some terms to logically equivalent forms) becomes:

Statement 8.2.

$$\forall x \forall z \forall z_2 \in \mathbb{N} :$$

$$\begin{aligned}
& [Z(0,0) \wedge \\
& \quad A(0,0) \wedge \\
& \quad B(0,0) \wedge \\
& \quad C(0,0) \wedge \\
& \quad \neg Z(x+1,z) \wedge \\
& \quad (Z(z,x) \implies Z(z,x+1)) \wedge \\
& \quad \neg(A(x,z) \wedge A'(x,z)) \wedge \\
& \quad (A(x+1,z) \implies A'(x,z)) \wedge \\
& \quad (A'(x+1,z) \implies A'(x,z)) \wedge \\
& \quad \neg(B(x,z) \wedge B'(x,z)) \wedge \\
& \quad (B(x+1,z) \implies B'(x,z)) \wedge \\
& \quad (B'(x+1,z) \implies B'(x,z)) \wedge \\
& \quad \neg(C(x,z) \wedge C'(x,z)) \wedge \\
& \quad (C(x+1,z) \implies C'(x,z)) \wedge \\
& \quad (C'(x+1,z) \implies C'(x,z)) \wedge \\
& \quad (C(z,x) \implies (C(z,x+1) \vee C^*(z,x+1)) \wedge \\
& \quad \quad (C^*(x,z) \implies C(x+1,z)) \wedge \\
& \quad (A(x,z) \implies \neg(C(x,z) \iff A^*(x+1,z))) \wedge \\
& \quad \quad (A^*(z,x) \implies A(z,x+1)) \wedge \\
& \quad \quad (B(z,x) \implies B^*(z,x+1)) \wedge \\
& \quad \quad (B^*(x+1,z) \implies B(x,z)) \wedge \\
& \quad (Z(z,x) \implies (B(z,x) \iff W(z,x))) \wedge \\
& \quad \quad (W(x,z) \iff W(x+1,z)) \wedge \\
& \quad \quad ((W(z,x) \wedge B(z,x)) \implies A(z,x+1)) \wedge \\
& \quad (C(z,x) \implies \neg(W(z,x) \iff C(z,x+1))) \wedge \\
& \quad \quad (C^*(x,z) \implies B(x+1,z))]
\end{aligned}$$

$$\begin{aligned}
& \wedge \\
& [(W(z, x) \implies (B(z, x) \iff D(z, x))) \wedge \\
& \quad (W(z, x) \implies \neg P(z, x)) \wedge \\
& \quad (A(x, z) \implies (A(x, z_2) \iff R_1(z_2, z))) \wedge \\
& \quad (B(x, z) \implies (B(x+1, z_2) \iff R_2(z_2, z))) \wedge \\
& \quad ((R_1(x, z) \wedge R_2(x, z)) \iff R_3(x, z)) \wedge \\
& \quad (R_3(z_2, z) \implies (D(x, z) \iff P(x, z_2))) \wedge \\
& \quad (W(x, z) \implies \neg U_1(x, z)) \wedge \\
& \quad (P(x, z) \implies (B(x, z_2) \iff U_1(z_2, z))) \wedge \\
& \quad ((R_1(x, z) \wedge U_1(x, z)) \iff U_2(x, z)) \wedge \\
& \quad (U_2(z, z_2) \implies (C(x, z) \iff D(x, z_2)))]
\end{aligned}$$

It is also worth noting that, one could theoretically drop the R_3 and U_2 predicates by replacing them with $(R_1 \wedge R_2)$ and $(R_1 \wedge U_1)$, respectively. For example,

$$R_3(z_2, z) \implies (D(x, z) \iff P(x, z_2))$$

can be expressed as

$$(R_1(z_2, z) \wedge R_2(z_2, z)) \implies (D(x, z) \iff P(x, z_2))$$

Similarly,

$$U_2(z, z_2) \implies (C(x, z) \iff D(x, z_2))$$

can be expressed as

$$(R_1(z, z_2) \wedge U_1(z, z_2)) \implies (C(x, z) \iff D(x, z_2))$$

and the clauses

$$(R_1(x, z) \wedge R_2(x, z)) \iff R_3(x, z)$$

and

$$(R_1(x, z) \wedge U_1(x, z)) \iff U_2(x, z)$$

can be dropped entirely.

Nonetheless, we will still use the R_3 and U_2 predicates to preserve elegance for later sections.

9 Expressing Constants

Because we are ultimately trying to prove that we can encode computation, it is worth a section on discussing constant terms. Constant terms are terms that are, loosely speaking, not defined by prior addition or multiplication operations or previous assignments. They serve as "base cases" for computation.

Terms that are computed from previous terms are called derived terms. We say "loosely" because as in section 9.2, we explain a way to compute individual constants using binary representations, that can reduce the number of predicates needed to express them. However, for our purposes, constants are interpreted to behave like constants in a computer program or formula.

For example, if we wanted to compute the Fibonacci numbers $\text{Fib}_0 = 0, \text{Fib}_1 = 1, \text{Fib}_n = \text{Fib}_{n-1} + \text{Fib}_{n-2}$, $\text{Fib}_0 = 0$ and $\text{Fib}_1 = 1$ would be considered constant terms, but $\text{Fib}_11 = 89$ is a derived term.

The bottom line is that encoding constants — especially without any prior computation, is expensive — either in terms of predicates, clauses, clause length, or both.

In this section, we describe multiple ways to implement constant terms, then the optimal choice / reasons for choosing the different methods. All logical statements below are extensions and of the context of Statement 8.2.

9.1 Enumerating Constants

Constant enumeration is one of two methods for calculating and storing constant values by counting up to a particular constant term n (i.e. $\{0, 1, 2, \dots, n\}$). In this section, we describe two ways of constant enumeration.

9.1.1 One Hot Encoding of Constants

Suppose we want to express the constant n , without doing any prior computation. A one-hot encoding of n counts all constants up to n : $\{0, 1, 2, \dots, n\}$, and requires a unique predicate for each of them. Suppose X_i is the predicate that encodes constant i . Let E denote the predicate that marks the end of the constant enumeration up to n . The one hot encoding of constant term n can be described as:

$$\begin{aligned}
& \forall x \forall z \in \mathbb{N} : \\
& ((W(x, z) \wedge A(x, z)) \implies I_0(x, z)) \wedge \\
& (I_n(z, x) \implies E(z, x + 1)) \wedge \\
& (E(z, x) \implies E(z, x + 1)) \wedge \\
& [\bigwedge_{i=0}^{n-1} (I_i(z, x) \implies I_{i+1}(z, x + 1))] \wedge \\
& [\bigwedge_{i,j=0,1,2,\dots,i \neq j}^n \neg(I_i(x, z) \wedge I_j(x, z))] \wedge \\
& [\bigwedge_{i=0}^n \neg(E(x, z) \wedge I_i(x, z))]
\end{aligned}$$

The first clause initiates the constant zero I_0 predicate. The second clause ensures that when the last constant predicate I_n is true for coordinate $(0, n)$, the end of the constant enumeration predicate E is true for $(0, n + 1)$. The third clause requires that the final end state infinitely propagate forward. The fourth clause propagates successive constant terms until I_n is reached. The fifth and sixth clauses ensure mutual exclusivity between constant predicate representations among themselves (i.e. I_0 and I_1 can't both be true) and the final end state (i.e. I_i and E can't both be true).

The number of clauses involved in the formula above is $3 + n + \binom{n+1}{2} + (n+1) = \frac{n^2+5n+8}{2}$.

In Big O complexity, we can say that the big O notation for the number of clauses relative to computing n via one-hot notation using the above formula is $O(n^2)$. For small constants (i.e. < 5 or 6), this may not be such a problem. However, for larger constants, this can cause an explosion in the number of clauses in the final First-Order representation of the constant value.

Since this is not very desirable, especially for moderately sized constants, it might be nicer to attempt to re-write the one hot formula with something equisatisfiable that doesn't cause a quadratic growth in the number of resulting clauses. Using the methods described in 6.1, we can re-write the formula instead using complementary auxiliary predicates for each constant enumerable predicate I_i as follows:

$$\begin{aligned}
& \forall x \forall z \in \mathbb{N} : \\
& ((W(x, z) \wedge A(x, z)) \implies I_0(x, z)) \wedge \\
& [\bigwedge_{i=0}^n (I_i(z, x) \implies E_i(z, x + 1))] \wedge \\
& [\bigwedge_{i=0}^n (E_i(z, x) \implies E_i(z, x + 1))] \wedge \\
& [\bigwedge_{i=0}^n \neg(I_i(x, z) \wedge E_i(x, z))] \wedge \\
& [\bigwedge_{i=0}^{n-1} (I_i(z, x) \implies I_{i+1}(z, x + 1))]
\end{aligned}$$

Here, E_i is a predicate that complements I_i to enforce that I_i is unique for each column.

The total number of clauses in this formulation is $1 + 3n + (n - 1) = 4n$. In Big O notation, this is linear with respect to n . The number of predicates involved is only double that of the predicates in the first formula (minus E), so it still remains linear in terms of n .

While this may be acceptable, say, for moderately sized constants, i.e. $n = 10, 1000, 10000$, it often becomes difficult if not in practice impossible for very large (i.e. 2^{512}) constants. We describe some encoding formulas in the next section to express the same constant enumeration using fewer predicates (in big O notation) to count up to n .

9.1.2 Bitwise predicate representation

In the previous subsection, we described a constant enumeration method that required each constant (starting from 0) to have its own unique predicate. The main advantage of this formulation is small clauses and fast unit propagation.

The second way of enumerating constants is to write each number up to n in binary, using $\lceil \log(n+2) \rceil$ bits [the extra count is for marking the end of the enumeration], writing each number in binary from left to right (i.e. Big-Endian notation). For example, to count to 9, we need the bit representations

$$\begin{aligned} 0 &= 0000_b \\ 1 &= 0001_b \\ 2 &= 0010_b \\ 3 &= 0011_b \\ 4 &= 0100_b \\ 5 &= 0101_b \\ 6 &= 0110_b \\ 7 &= 0111_b \\ 8 &= 1000_b \\ 9 &= 1001_b \end{aligned}$$

Additionally, we will need to count up one more value than n to mark the end of the constant enumeration, so we would also count to

$$10 = 1010_b$$

, where 10 would simply represent a state of non-enumeration.

The second step in deriving the formula then requires assigning each bit position a predicate. Suppose B_0 is the most significant bit and B_l is the least significant bit. In the example $n = 9$, l would be 3. Furthermore, denote $b(n)_l : \{0, 1\}^l$ to be the binary representation of n from left to right using $l > \lceil \log(n+2) \rceil$ bits (if $l < \lceil \log(n+2) \rceil$, to ensure b is well defined, we could simply define $b(n)_l$ as the bit representation of $n \pmod{2^{l+1}}$... i.e. representing the rightmost $l+1$ bits of n).

For example, $b(0) = (0, 0, 0, 0)$ if using 4 bits, $b(7) = (0, 1, 1, 1)$ if using 4 bits. $16 = (0, 0, 0, 1, 0, 0, 0)$ if using 7 bits.

Additionally, $b(n, i)_l$ denote the i -th member of the sequence $b(n)_l$, starting from left to right (left is most significant bit, right is least significant bit).

Let

$$S(n) = \bigwedge_{i=0}^l \lambda(B_i, n)$$

where

$$\lambda(B_i, n) = \begin{cases} B_i & \text{if } b(n, i)_l = 1 \\ \neg B_i & \text{if } b(n, i)_l = 0 \end{cases}$$

$S(n)$ acts as a "predicate" representation of constant n using l bits (assuming l is large enough).

For the example counting up to $n = 9$ using 4 ($l = 3$) bits, we have

$$\begin{aligned} S(0) &= \neg B_0 \wedge \neg B_1 \wedge \neg B_2 \wedge \neg B_3 \\ S(1) &= \neg B_0 \wedge \neg B_1 \wedge \neg B_2 \wedge B_3 \\ S(2) &= \neg B_0 \wedge \neg B_1 \wedge B_2 \wedge \neg B_3 \\ S(3) &= \neg B_0 \wedge \neg B_1 \wedge B_2 \wedge B_3 \\ S(4) &= \neg B_0 \wedge B_1 \wedge \neg B_2 \wedge \neg B_3 \\ S(5) &= \neg B_0 \wedge B_1 \wedge \neg B_2 \wedge B_3 \\ S(6) &= \neg B_0 \wedge B_1 \wedge B_2 \wedge \neg B_3 \\ S(7) &= \neg B_0 \wedge B_1 \wedge B_2 \wedge B_3 \\ S(8) &= B_0 \wedge \neg B_1 \wedge \neg B_2 \wedge \neg B_3 \\ S(9) &= B_0 \wedge \neg B_1 \wedge \neg B_2 \wedge B_3 \\ S(10) &= B_0 \wedge \neg B_1 \wedge B_2 \wedge \neg B_3 \end{aligned}$$

We then would apply the same formulation as in the previous section (without the auxiliary uniqueness predicates) for obtaining the enumeration clauses.

$$\forall x \forall z \in \mathbb{N} :$$

$$\begin{aligned} ((W(x, z) \wedge A(x, z)) &\implies S(0)(x, z)) \wedge \\ (S(n)(z, x) &\implies E(z, x + 1)) \wedge \\ (E(z, x) &\implies E(z, x + 1)) \wedge \\ (S(n + 1)(z, x) &\implies S(n + 1)(z, x + 1)) \wedge \end{aligned}$$

$$[\bigwedge_{i=0}^n (S(i)(z, x) \implies S(i+1)(z, x+1))] \wedge$$

It should be noted, of course, that by DeMorgan's Laws, the consequent of implications that involve $S(i)$ would have to be expanded out to l different conjunctive clauses (so that each clause is a pure disjunction). Additionally, E isn't strictly needed, though it acts as a convenient placeholder for the $S(n+1)$ predicate to quickly reference enumeration endings

The number of clauses needed in the above formulation would be $l+1+1+l+(n+1)l = nl+3l+2$. In Big O complexity, because $l = O(\log_2(n+2)) = O(\log n)$, the number of clauses for enumerating up to constant n has big O complexity $O(n \log n)$.

This is more acceptable than the initial $O(n^2)$ complexity for one-hot encodings and not significantly worse than linear complexity for one hot encodings using auxillary predicates.

Unit propagation can still be very efficient here, because at any step in the enumeration, all predicate bits $(B_0, B_1, \dots B_l)$ have a determined truth value for (z, x) , and by the implications in the formulation above, the next sequence truth values for $(B_0, B_1, \dots B_l)$ for the coordinate $(z, x+1)$ is determined, without additional constraints or ambiguity.

9.1.3 Local vs Global Constants and Variable reuse

9.2 Binary Encoding of Constants

9.3 Computing Constants

10 References

[TODO] @bookRogers1987, author = Rogers, Hartley, title = Theory of Recursive Functions and Effective Computability, publisher = MIT Press, year = 1987, address = Cambridge, MA